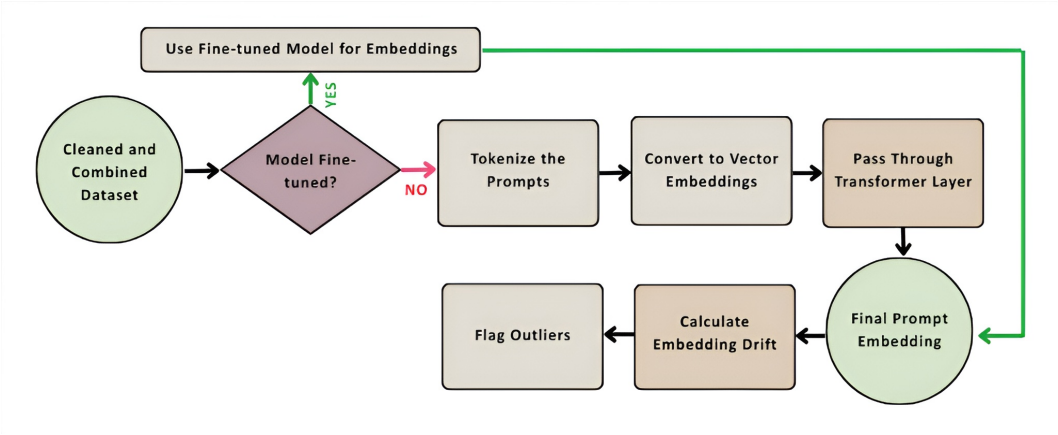

Figure 1: Overview of the ZEDD pipeline

4.1 Embedding Extraction

For each prompt in our matched clean/injected pairs (described in section F.1), we extract a vector representation using fine tuned embedding representations from **Sentence BERT All MPNET Base V2, Llama 3 8B Instruct, Mistral 7B Instruct, and Qwen 2 7B Instruct**. Further information on URLs and Licensing can be found in Appendix A.

During fine-tuning, the models utilize the embedding representations of each clean-injected and clean-clean prompt pair to better classify and identify the differences between injected and clean prompts in the embedding space, allowing for ZEDD to perform significantly better.

4.2 Drift Measurement and Detection

To quantify how adversarial prompts alter a model's internal understanding, we measure the **semantic drift** between each injected prompt and its clean counterpart utilizing **Cosine Similarity**. Using vector embeddings extracted from a language model's encoder, we compute cosine distance as a proxy for semantic change. A larger distance implies a greater shift in meaning, potentially indicating injection. This approach is significantly more lightweight in comparison to other approaches mentioned in Section 1.

We define this embedding drift score as:

$$Drift(x, x') = 1 - \frac{f(x) \cdot f(x')}{||f(x)|| \cdot ||f(x')||} \tag{1}$$

This formulation captures how much the injected prompt deviates from its clean counterpart, but is significantly more lightweight in comparison with previous approaches mentioned in Section 1.

In order to properly analyze our dataset of prompts, we separate our dataset into a training and testing dataset with around **70%** being the training dataset containing both fully clean (clean - clean) prompt pairs and partially clean (injected-clean) prompt pairs, keeping accurate category distributions to the original dataset for the injected-clean prompt pairs. We then run a **Binary Classification Evaluation** where the models we test get fine-tuned based on the category of the prompt pair, where the score is 1 if the category is clean and 0 otherwise, to properly establish a baseline where the model can learn the relationships of the prompts to optimize the way they are embedded. We used approximately **10%** of the training dataset (which as we recall was **70%** of the total dataset) to fine tune the models tested in an effort to reduce fine-tuning time and to prevent overfitting of the models.

4.3 Drift Detection Framework

To accurately detect adversarial prompt injections without labeled ground truth, we develop an **ensemble flagging approach** to classify suspected injected prompt pairs utilizing the drift scores of embeddings from each of the models.

4.3.1 Distributional Modeling and Threshold Calibration

Utilizing a hierarchical approach, our flagging algorithm first uses **Gaussian Mixture Modeling (GMM)** and has **Kernel Density Estimation (KDE)** as a fallback mechanism.

Gaussian Mixture Modeling (GMM): The system fits a two-component GMM on the drift-score distribution, using mean separation to separate clean and injected drift score populations, with the lower mean score corresponding to the clean-clean prompt pairs as they have lower semantic drift and the higher mean score corresponding to the injected-clean prompt pairs with higher semantic drift.

The optimal decision threshold is computed as:

$$f_{clean}(x) \cdot w_{clean} = f_{injected}(x) \cdot w_{injected} \tag{2}$$

Where $f_i(x)$ represents the Gaussian density function for component i and w_i denotes the mixture weight.

KDE Fallback: When GMM fails to converge or produces unstable results, the flagging algorithm falls back to a **KDE-based approach**, identifying peaks and valleys in the distribution to distinguish between the injected-clean and clean-clean prompt pairs.

4.3.2 Constrained Optimization for Detection Performance

The threshold optimization section aims to optimize both the false postive rate and the overall number of items flagged. These values are preset to values of **3%** and **50%** respectively as we found those to yield the most optimal performance, but have the possibility to be modified as needed.

The final threshold used to flag values is determined through **iterative binary search** within the feasible range, bounded by the statistical tail of the estimated clean distribution at the desired false positive rate in order to ensure that our threshold calculations can be applicable across embedding distributions.

5 Experimentation and Results

To ensure reproducibility and transparency we specifically fine-tuned each model utilizing the **NVIDIA B200 GPU from Runpod,** with hyperparameters available in the GitHub mentioned in the abstract. The fine-tuning times were approximately **15-18** minutes for each of the four models tested.

We executed the drift detector on a held-out test slice of **51,603** aligned pairs: **25,801** clean–clean and **25,802** injected–clean spanning five attack categories. Pairs were encoded in batches of `64` and scored with **cosine drift** (1–cosine similarity). The decision threshold was **selected automatically** via a 2-component GMM on the drift scores with a *clean false-positive cap* of 3% and a soft target of $\approx 50\%$ overall flagged rate.

Observations: High precision with a very low clean FPR **(2.93% avg across all models tested)** indicates the cap-controlled operating point is conservative on false alarms. Across models, slight weaknesses in classification were noticed within the **Jailbreak**, **Encoding Manipulation**, and the **System Leak** Categories. This dip in classification was most drastic within the **Sentence BERT Model**. However, from an overall standpoint, model performance in most categories had lower and upper bounds being primarily above 90% overall as shown in Table 4, showcasing effectiveness with the GMM and KDE flagging algorithm.

Table 1: Results by Category Distribution: Side-by-side comparison of ZEDD's performance on different model encoding types. In the the table headings, the percentage refers to the percent of entries flagged in the category. **"C"** refers to Clean, **"EM"** refers to encoding manipulation, **"J"** refers to Jailbreak, **"PC"** refers to Prompt Confusion, **"SL"** refers to System Leak, and **"TO"** refers to Task Override.

Model	**% C**	**% EM**	**% J**	**% PC**	**% SL**	**% TO**
Sentence BERT (All-MPNET-BASE-V2)	1.7%	95.9%	86.2%	90.5%	91.6%	86.7%
Llama 3 8B Instruct	5.5%	98.1%	92.2%	94.4%	96.7%	90.7%
Mistral 7B Instruct	2.3%	98.1%	92.2%	93.3%	96.9%	90.8%
Qwen 2 7B Instruct	2.2%	98.2%	90.8%	94.2%	96.8%	90.3%

Table 2: Side-by-side metrics at each model's unsupervised operating point (same cap and selection logic).

Encoder	**Acc.**	**Prec.**	**Recall (adv)**	**F1**	**Clean FPR**
SBERT All-MPNET-Base-V2	90.75%	99.65%	81.78%	89.84%	1.7%
Llama-3 8B Instruct	95.32%	95.85%	94.75%	95.30%	5.5%
Mistral 7B Instruct	95.55%	96.58%	94.45%	95.50%	2.3%
Qwen2-7B Instruct	95.46%	96.27%	94.52%	95.38%	2.2%

In comparison to other projects on Prompt Injection Classification, ZEDD outperforms existing models in many key areas such as **precision and F1 score** as shown in Figure 2. In addition these results (around 51,000 testing prompt pairs) were obtained after fine tuning within less than **8 minutes** on the NVIDIA B200 GPU on Runpod, showing a strong classification speed in combination with high accuracy.

6 Limitations and Future Works

Though ZEDD does pose good results, there are possible improvements to be made. The nature of ZEDD itself does have a reliance on the created embedding to properly measure drift and characterize injected prompts, which could pose limitations as smaller and larger LLMs utilize different semantic embedding types. The drift quality is directly tied to the embedding model that is chosen which could pose limitations in certain cases where the embedding model is not able to effectively capture the semantic meaning of prompts in its embedding space. In terms of scalability, there are methods in which the ZEDD model may run more efficiently with at a higher-scale, considering both more data and larger models to fine-tune.

In future works, we plan to address issues with size of the model by utilizing adaptive approaches to effectively conserve resources and compute better drift overall by adjusting for possible changes due to the size of the model in the semantic embedding space. In addition, it may be valuable to explore a Few-Shot method to improve ZEDD's accuracy, however it may compromise the lightweight, fast nature which ZEDD excels in, especially in larger datasets. We also plan to utilize multiple datasets with varying formats to ensure ZEDD stays effective on data not necessarily only in email form like LLMail-Inject is.

Because of the lightweight nature of ZEDD, there is a tradeoff with the fact that more injected prompts may bypass ZEDD potentially creating issues with injected prompts. There may also be cases where prompts are purposefully manipulated to bypass ZEDD on the embedding level. Because of this, we advocate ZEDD as a strong first defense against prompt injections due to its lightweight nature, but in future works, we plan to explore further how we can make ZEDD even tougher to bypass and increase accuracy.